

Milestone 1 — Bank Statement Analysis → P&L + Balance Sheet

Build Guide for the VS Code Agent

This guide is written for an AI coding agent (Claude in VS Code) to implement Milestone 1. It is based on the client's **actual** delivered files, whose real structure has been inspected and is documented below. Build against these exact column names, sheet names, and formula patterns — do not guess.

1. Goal in one sentence

Take a client's bank statement (PDF, CSV, or Excel), turn every transaction into a clean row, use Claude to categorise each one, assign the correct UK financial year, append the rows into the existing **RAW** tab of the client's workbook in its exact column layout, and refresh the **Analysis** pivot + P&L / Balance Sheet so the numbers recalculate with zero errors.

2. The three files the client gave us (and what each is for)

All three are in the project's `(samples/)` folder. They are the ground truth.

A. `Bank_summarised_Behesta_v1_2025.xlsx` — the master template the output must match. It has three sheets:

- `RAW (2)` — the main transactions ledger (1,500+ rows). **This is the tab new transactions get appended to.**
- `RAW (3)` — a second ledger with the **same column layout** as RAW (2). Treat it as the same format.
- `Analysis 24 Raw (2)` — the pivot/summary that rolls RAW up by category and financial year, with a P&L (PL) vs Balance Sheet (BS) split.

B. `May_24.csv` — an example of a CSV bank export from the client's bank (Barclays). This is one of the input formats the tool must read.

C. `Statement_03-MAY-24_AC_23996530__05073028.pdf` — an example Barclays PDF statement. The other input format the tool must read.

Note: the CSV and the PDF cover the **same period** (April-May 2024, account 23996530), so they're a perfect cross-check — parsing either should produce the same set of transactions.

3. EXACT structure of the RAW tab (build to this)

RAW (2) and RAW (3) share this header row (row 1). Columns A-M:

Col	Header	Meaning	How to fill it
A	No	Sequential row number	Running integer, continues from last existing row
B	SA	Financial year label, e.g. FY22, FY24	Derived from date (UK FY, see §5)
C	ACC	FY in 21/22 style	Derived from same FY
D	Date	Transaction date	Real Excel date (datetime), not text
E	Subcategory	Bank's own transaction type	From the source (e.g. "Contactless Card Purchase") — may be blank
F	Memo	Description / payee	The cleaned transaction description
G	Paid in	Money in (positive)	Amount if credit, else blank
H	Paid out	Money out (positive)	Absolute amount if debit, else blank
I	Balance	Running balance from bank	From statement if available, else leave blank
J	NET	Net of the row	Formula: $=G\{\text{row}\}-H\{\text{row}\}$
K	Balance UC	Cumulative balance	Formula: first row $=J\{\text{row}\}$, then $=K\{\text{prev}\}+J\{\text{row}\}$
L	Check UC	Reconciliation check	(present in template; match its existing pattern)
M	UC Category	The category	This is what Claude assigns (see §4)

Critical formula rules (match the template exactly):

- Column J is always $=G\{\text{row}\}-H\{\text{row}\}$.
- Column K chains: $=K\{\text{previous_row}\}+J\{\text{row}\}$ (the very first data row uses $=J\{\text{row}\}$).
- Use real Excel formulas as strings via openpyxl — never hardcode the computed numbers. After writing, run the recalc step (see §8).

4. The category list (what Claude must choose from)

The accountant uses a **fixed vocabulary** of categories in column M. The AI must pick from this list — not invent new ones. These are the real categories pulled from the template:

Accountancy, Bank charges, Car Insurance, Charging, Company Car, DLA, Directors salary, Donation, Entertainment, Equipment, Gym, HMRC, In/Out, Income, Insurance, Interest income, Investment, Lunch, Mobile phone, Mother Salary, Parking, Penalty fee, Petrol, Postage, Professional, Professional fees - College, Refund, Subscription, Sundry, Taxes for mother, Taxi, Train, Travel

Notes for the agent:

- The raw template has casing/spelling duplicates (Lunch/lunch/Lunch?, Company Car/Company car, Mother Salary/Mother salary). **Normalise to one canonical spelling each** (use the Title Case version) so the pivot doesn't split a category in two.
- Income is the main credit category. DLA (Director's Loan Account), HMRC, Investment map to the Balance Sheet; most others map to P&L (see §6).
- If Claude is genuinely unsure, it should return Sundry or Unknown rather than guessing wildly — a human reviews these.

5. UK financial year logic

The UK tax year runs **6 April to 5 April**. For any transaction date:

- If the date is on/after 6 April of year Y, it belongs to FY Y/Y+1.
- If before 6 April, it belongs to FY Y-1/Y.

Produce both label formats the template uses:

- SA column (B): FY + last two digits of the **start** year. E.g. a 2 May 2024 transaction → FY runs Apr 2024–Apr 2025 → FY24.
- ACC column (C): start/end short year, e.g. 24/25.

Edge case to handle: the boundary days (5 vs 6 April) must be correct. Test with a 5 April and a 6 April date.

6. The Analysis tab (pivot + P&L / Balance Sheet)

`Analysis 24 Raw (2)` is a pivot of the RAW data:

- **Rows** = category (column A, "Row Labels").
- **Columns** = financial year (`21/22`, `22/23`, `23/24`, ...).
- **Values** = `Sum of NET` for that category × year.
- Column E tags each category as `PL` (Profit & Loss) or `BS` (Balance Sheet) — this is how the same pivot feeds both statements.
- Column F holds a "presentation name" mapping the raw category to the accountant's P&L wording (e.g. raw `Lunch` → `Subsistence`, raw `Parking`/`Travel` → `Travel`).

What the agent should build:

- Regenerate this pivot from the RAW data using live formulas (`SUMIFS`) by category and FY is the natural choice, so it recalculates when RAW changes). Match the existing PL/BS tags and presentation-name mapping in columns E/F.
 - The **P&L** is the subset of rows tagged `PL`; the **Balance Sheet** is the subset tagged `BS`.
 - **Balance Sheet caveat:** a full Balance Sheet needs **opening balances** that live in the template, not in the bank data. Build the scaffold and pull opening balances from the template's existing cells — flag clearly in code where they come from.
-

7. Reading the two input formats

CSV (`May_24.csv`) — columns are: `Number, Date, Account, Amount, Subcategory, Memo`.

- `Amount` is **signed**: negative = money out, positive = money in. Split into Paid in (G) / Paid out (H) accordingly (store H as a positive number).
- `Date` is `D/M/YYYY` (e.g. `2/5/2024`) — parse day-first.
- `Memo` contains the payee plus a tab and a reference; clean it (strip tabs, collapse spaces) for column F. Keep `Subcategory` for column E.
- The file has ~48 transaction rows.

PDF (Barclays statement) — a multi-page table with columns: Date, Description, Money out £, Money in £, Balance £.

- Dates appear once per day and apply to following rows until the next date — carry the date down.
- Descriptions wrap across two lines — join them.

- Skip non-transaction lines: "Start Balance", "Balance brought forward", "Balance carried forward", "Total Payments/Receipts", and the "Helpful Information" pages at the end.
- Use the printed running Balance column to fill column I when present.
- **Privacy:** do NOT send account number, sort code, IBAN, or address to the AI. Only the description/date/amount go to Claude (see §9).

Both parsers must output the **same internal transaction shape**: `{date, description, subcategory, money_in, money_out, balance}`. Everything downstream works off that shape, so adding a new bank format later is just a new parser.

8. Build sequence for the agent

1. **Project skeleton** — `(parsers/)` (csv, pdf), `(core/)` (normalise, financial_year, categorise, excel_writer), `(samples/)`, `(output/)`, `(.env)` for the API key, `(requirements.txt)`.
 2. **Parsers** — write `(parse_csv())` and `(parse_pdf())`, each returning the common transaction shape. Test both against the samples; they should agree.
 3. **Financial-year module** — implement §5 with unit tests on the 5/6 April boundary.
 4. **Categorisation module** — §4 and §9. Batch transactions to the API; cache results.
 5. **Excel writer** — load `(Bank_summarised_Behesta_v1_2025.xlsx)` with openpyxl, append rows to `(RAW (2))` in the exact A-M layout with the J/K formulas, then (re)build the Analysis pivot with SUMIFS formulas.
 6. **Recalculate** — run LibreOffice recalc (`(python scripts/recalc.py output.xlsx)`) and confirm **zero** formula errors (`#REF!`, `#DIV/0!`, `#VALUE!`, `#NAME?`). Fix any and repeat.
 7. **Verify** — check the appended rows' running balance (column K) matches the bank's printed end balance (£20,278.51 for the sample period). If they match, parsing + maths are correct.
-

9. How the AI categorisation should work

- Model: **Claude Haiku** (cheapest; this is simple classification). Use the client's API key from `(.env)` — never hardcode it.
- Send **only** the cleaned description, the subcategory, and the amount sign (in/out). Never send account numbers, sort codes, names of account holders, or addresses.
- Prompt pattern: give Claude the fixed category list from §4 and ask it to return the single best-matching category per transaction, as structured output (JSON array), nothing else.

- Batch many transactions per request to keep cost to pennies. Cache by description so repeated payees (e.g. "TFL TRAVEL CH") aren't re-sent every month.
 - Have a small hardcoded rule layer in front of the AI for obvious repeat payees (TFL → Travel, salary standing orders → Directors/Mother salary), so the AI only handles the genuinely ambiguous ones. This cuts cost and increases consistency.
-

10. Definition of done

- Feed in `May_24.csv` → rows appear in `RAW (2)` with correct FY, in/out split, formulas, and sensible categories.
 - Feed in the Barclays PDF for the same period → produces the same transactions.
 - Account/sort/IBAN never leave the machine.
 - Analysis pivot, P&L, and Balance Sheet recalc with **zero** formula errors.
 - Running balance reconciles to the statement's end balance.
 - API key read from `.env`; no secrets in code.
-

11. Tech stack (install these)

- **Python 3**
- **pandas** — CSV/Excel reading and the transaction table
- **pdfplumber** — extract the transaction table from the Barclays PDF (try this first; fall back to **camelot** only if table extraction struggles)
- **openpyxl** — append to the template and write live formulas (preserves the existing sheets)
- **anthropic** (official SDK) — Claude Haiku categorisation
- **python-dotenv** — load the API key from `.env`
- **LibreOffice** — for the formula recalculation/verification step

`requirements.txt`:

```
pandas
pdfplumber
openpyxl
anthropic
python-dotenv
```

12. Things NOT to do

- Don't hardcode calculated totals — use Excel formulas so the workbook stays live.
- Don't impose new formatting on the template — match its existing style, headers, and conventions exactly.
- Don't send any account-identifying data to the AI.
- Don't invent new category names — use the fixed list and normalise duplicates.
- Don't commit the `.env` or the API key to git.